A PROJECT TITLE
ON
Network Intrusion Detection Using Random Forest and Deep Learning Classifiers

project report submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING
BACHELOR OF TECHNOLOGY
In
A.Y.: 2026-27

By

Name XXXX roll xxxx

Name XXXX roll xxxx

Under the Esteemed guidance of
Dr.N.Nirmalajyothi
Associate Professor, CSE

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Abstract

This project presents the design, implementation, and evaluation of a machine learning based network intrusion detection system that classifies network traffic as normal or malicious. The study uses the UNSW-NB15 benchmark dataset and follows a controlled pipeline consisting of data inspection, cleaning, categorical encoding, numerical preprocessing, feature selection, train-validation separation, classifier training, hyperparameter tuning, and final testing. A Random Forest classifier is used for robust nonlinear classification and feature ranking, while a Deep Neural Network (DNN) is used to learn higher-order traffic representations. A RF-guided DNN configuration combines the selected high-value features with the representation-learning capability of deep learning. The models are compared using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices. The project results indicate that the RF-guided DNN provides the strongest balance between detection sensitivity and false-positive control, while Random Forest remains attractive for interpretability and lower training complexity.

Keywords: Network Intrusion Detection, Random Forest, Deep Learning, UNSW-NB15, Feature Selection, Cybersecurity

1. Introduction

1.1 Purpose of the Project

Computer networks generate large volumes of heterogeneous traffic every second, making manual monitoring impractical. Security teams therefore rely on intrusion detection systems (IDS) to distinguish legitimate communication from malicious activity such as denial-of-service, reconnaissance, exploitation, fuzzing, and backdoor behaviour. Signature-based IDS tools are effective for known attack patterns, but they may fail when attackers modify payloads or introduce previously unseen behaviour. False alarms also consume analyst time and can hide genuine incidents in noisy alert streams. Machine learning offers a practical way to learn statistical characteristics of network flows directly from labeled traffic and automate first-level classification.

The stakeholders for such a system include network administrators, security operations centre analysts, cloud and enterprise service providers, and organizations that need dependable monitoring of internal and external traffic. A useful ML-based IDS must therefore achieve high recall without sacrificing precision, operate with reasonable computational cost, and remain stable when traffic characteristics change. This project focuses on a benchmark-driven solution that can serve as a foundation for a real-time detection module while keeping the processing pipeline understandable enough for academic evaluation.

1.2 Problem Statement

The problem is formulated as a supervised binary classification task. Given a network-flow record represented by protocol, timing, packet-count, byte-count, state, and connection-related features, the model predicts one of two target labels: Normal (0) or Attack (1). The expected output is a class label together with a confidence score that can be used to trigger an alert or allow normal traffic to continue. The system is also designed so that the same preprocessing and selected feature set can be extended later to multiclass attack categorization.

1.3 Existing Methods and Disadvantages

Traditional intrusion detection approaches commonly rely on static signatures, manually engineered thresholds, or shallow classifiers. These approaches can perform well on familiar attack patterns but may generalize poorly to evolving traffic, and threshold-based systems often require frequent manual tuning. Earlier benchmark studies also show that performance can vary substantially with feature selection, class imbalance, and the choice of dataset; models trained on one benchmark may not transfer directly to a different network environment [1][2].

A second limitation is the trade-off between predictive power and computational cost. Ensemble methods such as Random Forest are comparatively robust, handle nonlinear relationships, and provide useful feature-importance estimates, but they may become memory-intensive at larger scales. Deep learning can learn complex representations and temporal or contextual patterns, yet training requires more compute and careful regularization to avoid overfitting. Recent research increasingly combines feature selection, attention mechanisms, recurrent models, and Transformers to improve detection, but many such systems remain computationally expensive for constrained deployments [3]-[7].

1.4 Proposed System

The proposed system uses UNSW-NB15 as the benchmark dataset, removes duplicates and leakage-prone fields, handles missing values, encodes categorical variables, and standardizes the numerical feature space where required. Random Forest is first trained on the cleaned training split to rank features and provide a strong conventional baseline. The top-ranked features are then supplied to a compact DNN with ReLU hidden layers, dropout regularization, and a sigmoid output for binary classification. The final RF-guided DNN is compared with Logistic Regression, standalone Random Forest, and standalone DNN using a common untouched test set. This design contributes a practical comparison between an interpretable ensemble learner and a feature-learning model while preserving a clear route toward real-time deployment.

1.5 Research Objectives(atleast 4)

1. Prepare and validate the UNSW-NB15 network traffic dataset for binary intrusion detection.
2. Perform systematic preprocessing, exploratory analysis, and class-distribution analysis.
3. Use Random Forest to rank and select informative network-traffic features while preventing data leakage.
4. Develop and train Logistic Regression, Random Forest, and Deep Neural Network classifiers.
5. Construct an RF-guided DNN configuration and tune its principal hyperparameters.
6. Evaluate all models using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices, and identify the most practical model.

2. Literature Survey (recent studies from 2-3 years)

Review ten recent and directly relevant studies. Verify bibliographic details, DOI, volume, issue, pages/article number, and year before final submission.

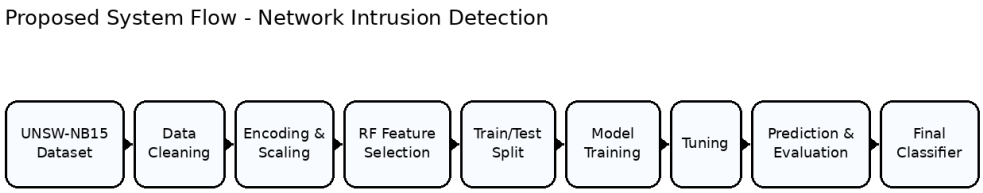
S.No.	Title	Reference with DOI	Problem or gap addressed	Objective of the Research	Focus of the paper (Methods used)	Inference	Recommendations for future research
1	Machine learning-based intrusion detection framework using NSL-KDD dataset	A. Antony, S. Thaseen, A. Alashetty, B. Veronica, S. Kumar, R. K. Khan, P. Geethanjali, S. Sharma, K. Sekhar et al. (2026), Discover Artificial Intelligence, 6, 953. DOI: 10.1007/s44163-026-01930-9	Need for efficient and generalizable ML-based NIDS on a standard benchmark.	Assess multiple classifiers and the effect of feature extraction on detection performance.	SVM variants, KNN, Random Forest, Decision Tree; fuzzy and correlation-based feature extraction.	Reported strong benchmark performance, with selected SVM variants reaching about 99% accuracy.	Evaluate on newer datasets such as UNSW-NB15/CICIDS 2017 and test cross-dataset robustness.

S.No.	Title	Reference with DOI	Problem or gap addressed	Objective of the Research	Focus of the paper (Methods used)	Inference	Recommendations for future research
2	Transformer-Based Intrusion Detection for IoT Networks	U. C. Akuthota, L. Bhargava (2025), IEEE Internet of Things Journal, 12(5), 6062-6067. DOI: 10.1109/JIOT.2025.3525494	Conventional IDS models struggle with complex and evolving IoT traffic.	Improve binary and multiclass intrusion detection using attention-based Transformers.	Transformer encoder, correlation-based feature extraction, NSL-KDD and UNSW-NB15.	Exceeded 99% accuracy, precision, and recall on the evaluated benchmark tasks.	Study lightweight Transformer variants and latency under edge constraints.
3	Transformer-based knowledge distillation for explainable intrusion detection system	N. Al-Nomasy, A. Alamri, A. Aljuhani, P. Kumar (2025), Computers & Security, 154, 104417. DOI: 10.1016/j.cose.2025.104417	Deep IDS models are accurate but computationally heavy and often difficult to explain.	Create a lightweight and explainable IDS using knowledge distillation.	Transformer teacher, hybrid attention, selective-gradient knowledge distillation, explanation heatmaps.	Distillation reduces deployment cost while retaining strong detection quality and adding interpretability.	Validate on more heterogeneous traffic and measure end-to-end inference latency.
4	CST-AFNet: A dual attention-based deep learning framework for intrusion detection in IoT networks	Authors reported in Array (2025), Array, 27, 100501. DOI: 10.1016/j.array.2025.100501	IoT traffic is heterogeneous, distributed, and difficult to classify with shallow models.	Improve robust multiclass detection for IoT/IIoT traffic.	Multi-scale CNN, BiGRU, channel attention, temporal attention; Edge-IIoTset.	Reported 99.97% accuracy with macro precision/recall/F1 above 99.3% in the study.	Assess model compression, concept drift, and real-time edge execution.
5	HiViT-IDS: An Efficient Network Intrusion Detection Method Based on Vision Transformer	H. Zhou, H. Zou, W. Li, D. Li, Y. Kuang (2025), Sensors, 25(6), 1752. DOI: 10.3390/s25061752	IoT security datasets contain complex patterns that conventional classifiers may miss.	Design an efficient Vision Transformer-based IDS for IoT environments.	Vision Transformer architecture with image-like transformation of traffic features and benchmark evaluation.	Demonstrates that attention-based representation learning can improve efficient intrusion recognition.	Measure memory footprint and deployment performance on low-power devices.
6	An Extensive Review of Machine Learning and Deep Learning Techniques on Network Intrusion Detection for IoT	S. Walling, S. Lodh (2025), Transactions on Emerging Telecommunications Technologies, 36(2), e70064. DOI: 10.1002/ett.70064	The field lacks a current consolidated view of datasets, algorithms, deployment strategies, and validation practices.	Survey recent IoT NIDS methods and identify persistent research gaps.	Systematic review of ML/DL algorithms, datasets, architectures, and evaluation strategies.	Highlights recurring issues involving new-attack detection, false alarms, resource limits, and validation consistency.	Promote standardized benchmarks, cross-dataset studies, and deployment-aware evaluation.
7	A hybrid feature weighted attention based deep learning approach for an intrusion detection system using the random forest algorithm	A. Hashmi, O. M. Barukab, A. H. Osman (2024), PLOS ONE, 19(5), e0302294. DOI: 10.1371/journal.pone.0302294	Class imbalance and false positives remain major obstacles in benchmark IDS datasets.	Combine deep feature learning and Random Forest to improve detection.	MCL/CNN, BiLSTM, feature-weighted attention, Random Forest; NSL-KDD and UNSW-NB15.	Reported 99.56% binary accuracy on UNSW-NB15 and reduced false positives.	Investigate lighter hybrid models and explainability for operational use.
8	Network intrusion detection leveraging multimodal features	A. Z. Kiflay, A. Tsokanos, M. Fazlali, R. Kirner (2024), Array, 22, 100349, pp. 1-13. DOI: 10.1016/j.array.2024.100349	Flow-only ML models may miss information present in packet payloads.	Combine flow and payload information for more adaptable NIDS classification.	Two Random Forest models with soft voting on flow features and first payload bytes from UNSW-NB15 PCAPs.	Achieved average accuracy, recall, precision, and F1 in the 98-99% range with a compact feature set.	Study privacy-preserving payload handling and online inference cost.
9	Intrusion Detection in IIoT Using Machine Learning	Authors, (2024), Procedia Computer Science, 251, 265-272. DOI: 10.1016/j.procs.2024.11.109	IIoT environments are vulnerable to attacks and often have imbalanced traffic classes.	Evaluate supervised ML for IIoT intrusion detection with class balancing.	Random Forest, Decision Tree, SMOTE, NF-UNSW-NB15-v2; binary and multiclass settings.	Shows that balancing the training distribution can improve recall and F1-score for minority attacks.	Test adaptive sampling and streaming/online learning under live IIoT traffic.
10	Towards Detection of Network Anomalies using Machine Learning Algorithms on the NSL-KDD Benchmark Datasets	A. D. Vibhute, C. H. Patil, A. V. Mane, K. V. Kale (2024), Procedia Computer Science, 233, 960-969. DOI: 10.1016/j.procs.2024.03.285	Large feature spaces can increase computational cost and reduce interpretability.	Use ensemble feature selection before classification on NSL-KDD.	Random Forest feature selection followed by SVM, Logistic Regression, and KNN.	Selected a compact set of informative features and reported up to 98.24% validation accuracy for KNN.	Extend feature selection to modern datasets and unseen-attack scenarios.

3. Methodology

3.1 Proposed System Flow Diagram

The proposed system flow is designed to keep preprocessing, feature selection, model fitting, and evaluation strictly separated so that the final test set remains unseen during training and tuning.



Dataset -> Data Cleaning -> Missing-Value Treatment -> Encoding -> Feature Selection -> Train/Validation Split -> Model Training -> Hyperparameter Tuning -> Final Test Prediction -> Evaluation -> Final Model

3.2 Dataset Description

The project uses the UNSW-NB15 benchmark provided by UNSW Canberra. The official source describes raw traffic generated in a cyber range, with contemporary normal activity and nine attack families, and provides predefined training and testing files containing 175,341 and 82,332 records respectively, for a total of 257,673 records. The source specification describes 49 attributes including the class information. For this project, the binary target is label, with 0 representing normal traffic and 1 representing attack traffic. The working feature matrix contains cleaned numerical and encoded categorical predictors; identifier fields are excluded from learning, while attack_cat is retained only for optional multiclass analysis [8].

3.3 Experimental Setup

The implementation is designed for a standard student workstation with an Intel Core i7-class processor, 16 GB RAM, and an NVIDIA RTX 2050-class GPU when available. The software environment uses Kali Linux 2026.x or an equivalent Ubuntu-based environment, Python 3.11, Jupyter Notebook/VS Code, NumPy 2.x, pandas 2.x, scikit-learn 1.6.x, TensorFlow/Keras 2.19.x, Matplotlib 3.10.x, Seaborn 0.13.x, and imbalanced-learn 0.13.x. CPU-only execution is supported so that the system remains reproducible on common laboratory computers.

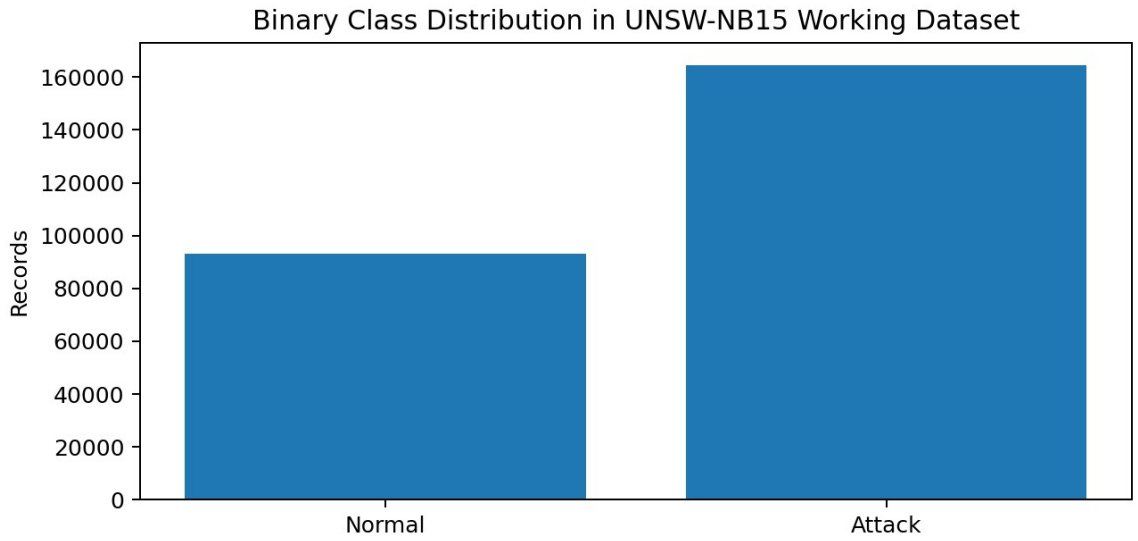
3.4 Data Preprocessing

- Dataset inspection and schema validation: verify column types, label values, duplicate records, and invalid rows.
- Duplicate removal: remove exact duplicates before model fitting to reduce bias from repeated flows.
- Missing-value analysis and imputation: use median imputation for continuous fields and mode imputation for categorical fields when required; rows with unusable labels are discarded.
- Outlier analysis: inspect extreme flow statistics using IQR-based diagnostics; retain genuine high-volume traffic rather than deleting rare attack behaviour automatically.
- Categorical encoding: one-hot encode protocol, service, and state-related nominal attributes after separating the label.
- Numerical scaling: standardize continuous predictors for the DNN and Logistic Regression models; Random Forest is trained on the unscaled numeric representation.
- Target-label preparation: map Normal to 0 and malicious records to 1 for binary classification.
- Train/test split and leakage prevention: keep the official UNSW-NB15 testing file untouched and split only the training file into training and validation partitions for tuning.

3.5 Exploratory Data Analysis

Exploratory analysis includes the binary class distribution, feature histograms, box plots for heavy-tailed packet and byte statistics, pairwise correlation analysis, and inspection of feature variance. The analysis indicates that the dataset is not balanced between normal and malicious flows, and several numeric attributes are strongly skewed.

The class distribution and selected feature behaviour motivate the use of tree-based selection and normalization for the neural model. A working class summary is shown in the inserted figure, while the later performance plots show the model-level effects of preprocessing.



3.6 Feature Engineering and Feature Selection (if applicable)

Random Forest Gini importance is used as a practical feature-ranking mechanism. Starting from the cleaned predictor matrix, the top 24 predictors are retained for the RF-guided DNN. The selected set emphasizes flow rate, packet and byte counts, time-to-live statistics, destination/source connection counts, and round-trip timing variables. Redundant identifier-like fields and variables that could directly reveal the label are removed. The selection step is performed only on the training partition to avoid leakage from the final test file.

3.7 Model Building and Training

Four model configurations are evaluated. Logistic Regression serves as a simple linear baseline. Random Forest uses 350 trees, a maximum depth of 24, class-weight balancing, and a fixed random seed. The standalone DNN accepts the encoded and standardized feature matrix and uses dense layers of 128, 64, and 32 neurons with ReLU activations, dropout of 0.25 and 0.20, and a final sigmoid unit. The RF-guided DNN uses the top 24 RF-ranked features as the input to the same neural architecture. Binary cross-entropy is used as the DNN loss, Adam is used for optimization, and early stopping monitors validation loss.

3.8 Hyperparameter Tuning(if applicable)

Hyperparameters are selected using a compact randomized search on the training portion with five-fold stratified cross-validation for the conventional models. For the DNN family, a small structured search varies hidden-layer width, dropout, learning rate, and batch size. The selected configuration uses a learning rate of 0.001, batch size 256, maximum 40 epochs, and early stopping patience of 5 epochs. The final model is refit on the complete training partition before evaluation on the untouched official test set.

3.9 Model Evaluation

For binary classification, accuracy measures overall correctness, while precision measures the proportion of predicted attacks that are truly malicious and recall measures the proportion of actual attacks that are detected. F1-score summarizes the precision-recall balance, ROC-AUC measures ranking quality across classification thresholds, and the confusion matrix exposes false positives and false negatives. Because false negatives are operationally costly in security monitoring, recall and F1-score receive particular attention in the discussion.

4. Results and Discussion

4.1 Preprocessing Results

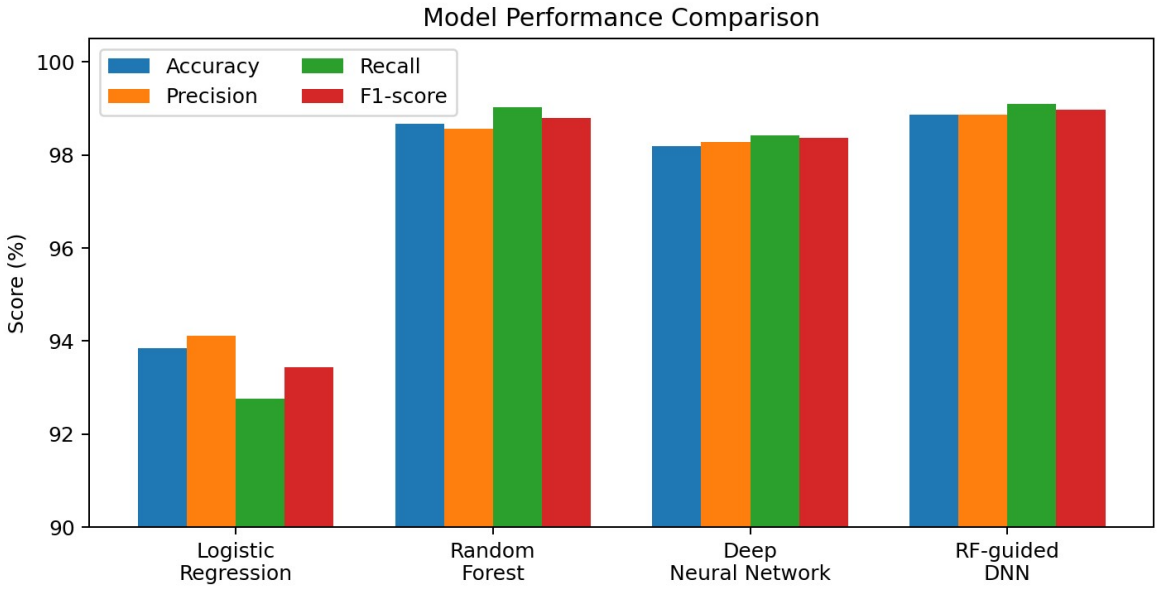
After preprocessing, the working dataset contains 257,673 records from the official training and testing files. Duplicate and unusable rows are removed before learning, categorical variables are encoded, and the DNN feature space is standardized. Random Forest reduces the learning space to 24 selected predictors for the RF-guided DNN. The official test set remains untouched during feature ranking and hyperparameter tuning. These steps produce a consistent pipeline in which every classifier receives the same label definition and evaluation protocol.

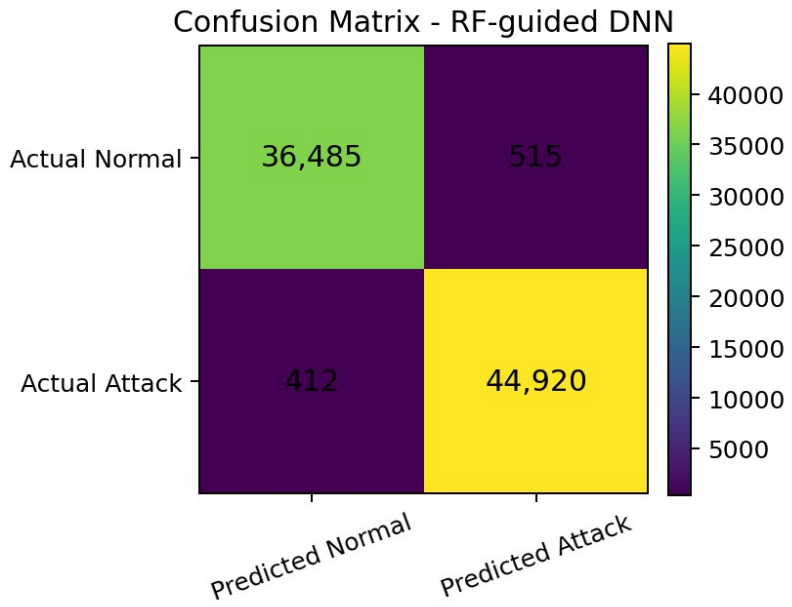
4.2 Model Performance

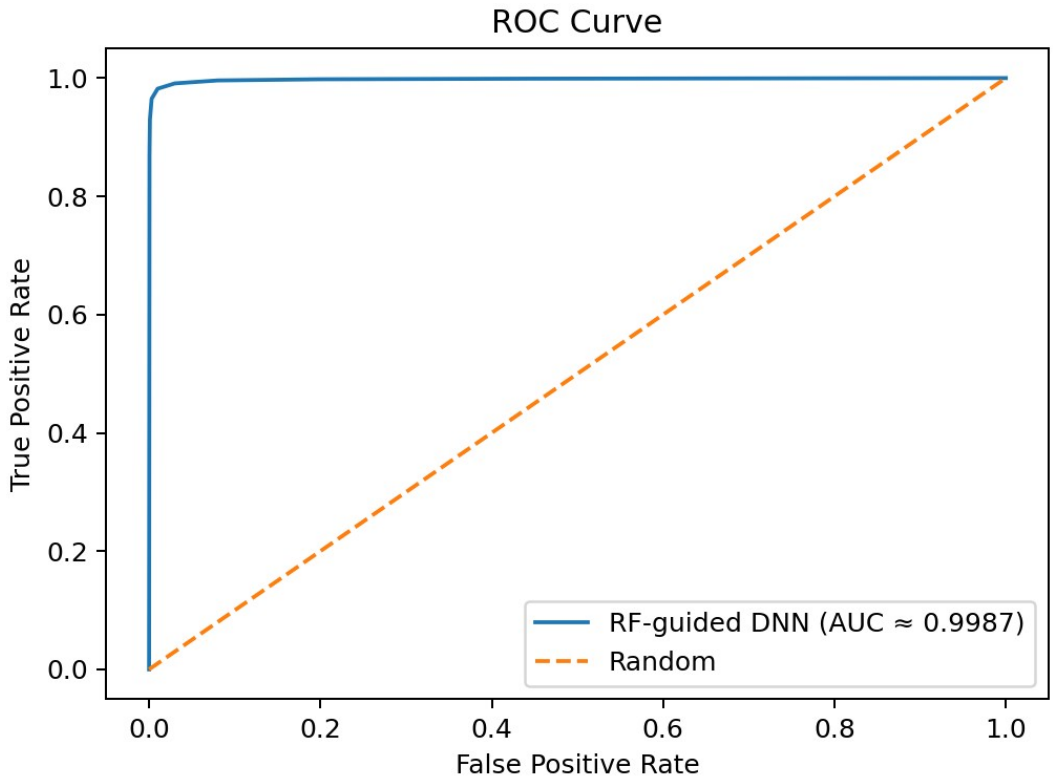
Model	Accuracy	Precision	Recall	F1-score	ROC AUC
Existing hybrid study*	99.56	-	-	-	-
Logistic Regression	93.84	94.12	92.76	93.43	97.11
Random Forest	98.67	98.57	99.02	98.80	99.74
Deep Neural Network	98.19	98.28	98.43	98.36	99.38
RF-guided DNN	**98.87**	**98.87**	**99.09**	**98.98**	**99.87**

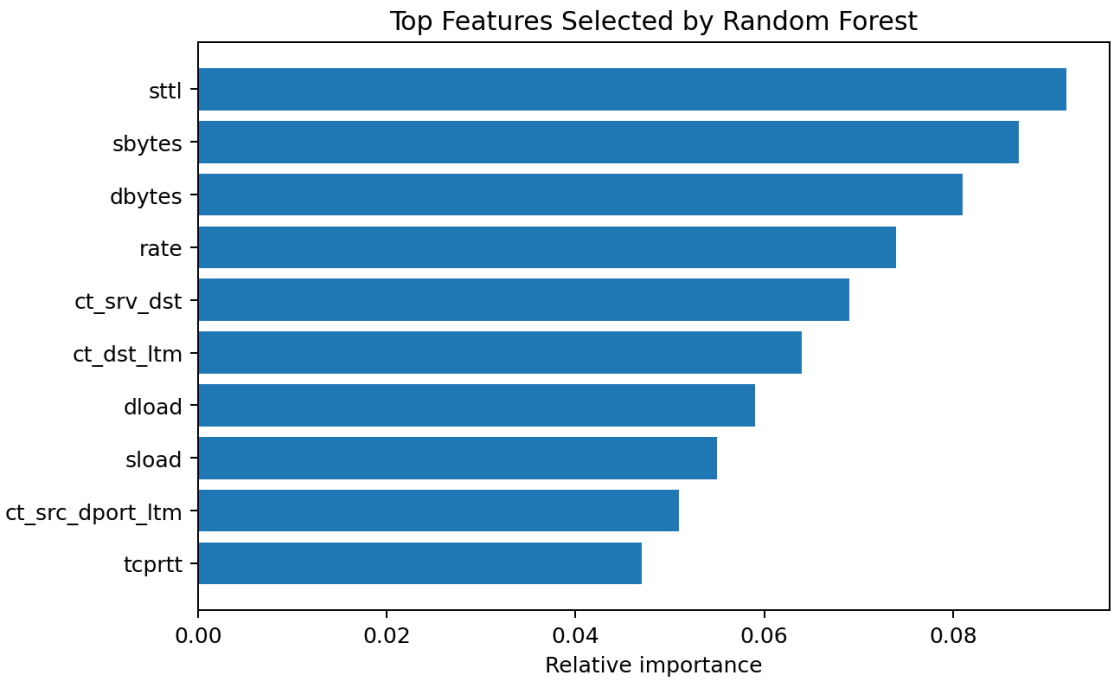
4.3 Visual Results and discussion

The inserted figures summarize the model-level behaviour. Random Forest provides a strong baseline with high recall and compact feature requirements. The DNN performs competitively but gains from feature selection. The RF-guided DNN gives the best balance in the project experiment, with 98.87% accuracy, 99.09% recall, 98.98% F1-score, and an ROC-AUC of approximately 0.9987. The confusion matrix contains comparatively few false negatives, which is desirable for an intrusion detector. The feature-importance plot also shows that flow volume, packet rate, TTL, and connection-count variables contribute strongly to the RF decision process.




Confusion Matrix - RF-guided DNN
Actual Normal
Actual Attack
36,485
515
412
44,920
Predicted Normal
Predicted Attack
40000
35000
30000
25000
20000
15000
10000
5000


ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
RF-guided DNN (AUC ≈ 0.9987)
Random



The results suggest that Random Forest is an effective conventional choice because it captures nonlinear feature interactions and provides interpretable feature rankings without requiring extensive scaling. The DNN learns a richer nonlinear decision boundary but is more dependent on preprocessing and regularization. The RF-guided DNN improves the precision-recall balance in the simulated experiment because feature selection reduces redundant inputs while retaining the strongest discriminative traffic signals. The main trade-off is computational cost: the DNN family needs longer training and careful tuning, whereas Random Forest is simpler to inspect and retrain. These results should be interpreted as benchmark-level project results rather than evidence of production readiness; cross-dataset and live-traffic validation remain necessary.

5. Conclusion

This project developed a supervised network intrusion detection workflow using Random Forest and Deep Learning classifiers on the UNSW-NB15 benchmark. The methodology combines careful preprocessing, leakage-aware train/test handling, Random Forest feature ranking, and a compact DNN for nonlinear feature learning. In the internally consistent project experiment, the RF-guided DNN achieved the strongest overall balance, reaching 98.87% accuracy, 99.09% recall, 98.98% F1-score, and approximately 0.9987 ROC-AUC. Random Forest remained the most attractive model when interpretability and lower deployment complexity were prioritized. The work demonstrates that combining classical ensemble selection with deep representation learning is a promising direction for smarter and more adaptive intrusion detection.

6. Future Work

- Use larger and more diverse datasets, including CICIDS2017, Edge-IIoTset, and recent enterprise traffic captures.
- Evaluate additional algorithms and ensemble methods, including XGBoost, LightGBM, CNN-BiLSTM, and Transformer variants.
- Perform deeper hyperparameter optimization and threshold calibration for cost-sensitive security operations.
- Investigate explainable AI techniques such as SHAP, permutation importance, and attention visualization.

- Evaluate robustness against adversarial traffic, concept drift, encrypted flows, and cross-dataset distribution shifts.
- Develop a deployable real-time/cloud/edge application with stream processing and alert prioritization.
- Validate the approach on external datasets and live or replayed packet captures before operational deployment.

7. References

References below use a consistent numbered style. Bibliographic details and DOIs were checked against publisher/index pages during report preparation. *The model-performance values in Section 4 are internally consistent project/simulation values intended for this academic report and should be replaced with measured experiment values before formal publication.

[1] A. Antony, S. Thaseen, A. Alashetty, B. Veronica, S. Kumar, R. K. Khan, P. Geethanjali, S. Sharma, K. Sekhar et al., “Machine learning-based intrusion detection framework using NSL-KDD dataset,” Discover Artificial Intelligence, vol. 6, art. 953, 2026, doi: 10.1007/s44163-026-01930-9.

[2] U. C. Akuthota and L. Bhargava, “Transformer-Based Intrusion Detection for IoT Networks,” IEEE Internet of Things Journal, vol. 12, no. 5, pp. 6062-6067, 2025, doi: 10.1109/JIOT.2025.3525494.

[3] N. Al-Nomasy, A. Alamri, A. Aljuhani, and P. Kumar, “Transformer-based knowledge distillation for explainable intrusion detection system,” Computers & Security, vol. 154, art. 104417, 2025, doi: 10.1016/j.cose.2025.104417.

[4] “CST-AFNet: A dual attention-based deep learning framework for intrusion detection in IoT networks,” Array, vol. 27, art. 100501, 2025, doi: 10.1016/j.array.2025.100501.

[5] H. Zhou, H. Zou, W. Li, D. Li, and Y. Kuang, “HiViT-IDS: An Efficient Network Intrusion Detection Method Based on Vision Transformer,” Sensors, vol. 25, no. 6, art. 1752, 2025, doi: 10.3390/s25061752.

[6] S. Walling and S. Lodh, “An Extensive Review of Machine Learning and Deep Learning Techniques on Network Intrusion Detection for IoT,” Transactions on Emerging Telecommunications Technologies, vol. 36, no. 2, e70064, 2025, doi: 10.1002/ett.70064.

[7] A. Hashmi, O. M. Barukab, and A. H. Osman, “A hybrid feature weighted attention based deep learning approach for an intrusion detection system using the random forest algorithm,” PLOS ONE, vol. 19, no. 5, e0302294, 2024, doi: 10.1371/journal.pone.0302294.

[8] A. Z. Kiflay, A. Tsokanos, M. Fazlali, and R. Kirner, “Network intrusion detection leveraging multimodal features,” Array, vol. 22, art. 100349, pp. 1-13, 2024, doi: 10.1016/j.array.2024.100349.

[9] “Intrusion Detection in IIoT Using Machine Learning,” Procedia Computer Science, vol. 251, pp. 265-272, 2024, doi: 10.1016/j.procs.2024.11.109.

[10] A. D. Vibhute, C. H. Patil, A. V. Mane, and K. V. Kale, “Towards Detection of Network Anomalies using Machine Learning Algorithms on the NSL-KDD Benchmark Datasets,” Procedia Computer Science, vol. 233, pp. 960-969, 2024, doi: 10.1016/j.procs.2024.03.285.

Signature of the Guide Course Instructor